Optimization Challenge: Movie Recommendation System

Link to optimized code: https://github.com/DinithiLiyanage/Movie-Recommendation-System

1. Scope

This report presents the performance analysis and optimization of a collaborative filtering based movie recommendation system. The system models user ratings using a movie × user matrix, where each entry represents a user's rating for a given set of movies. To account for differences in individual rating behavior, the ratings are normalized, producing a standardized user preference vector. Similarity between users is then computed using a Pearson correlation based approach applied to these normalized vectors. Based on the computed similarity scores, K-means clustering is used to identify groups of similar users, from which the most relevant neighbors are selected. For movies that a target user has not rated, the system predicts ratings using a weighted average of ratings from these similar users. This report evaluates the system's performance by examining the execution environment, profiling methodology, identified bottlenecks in the original implementation, applied optimizations, and the resulting performance improvements.

2. Execution Environment

The experiments were conducted under the following environments:

- **Host OS:** Windows
- **CPU: Intel(R) Core(TM) i7-1165G7 @ 2.80GHz**
- **Cores: 4**
- **Threads: 8**
- **Runtime Environment:** Windows Subsystem for Linux (WSL2)
- **Kernel:** Linux (WSL-provided kernel)
- **Compiler:** GCC 13.3.0

3. Profiling Tools and Techniques

3.1 Hardware-Level Profiling

The primary profiling tool used was **perf** which provides aggregate performance metrics:

```
perf stat -r 5 -- ./recommender
```

3.2 Function-Level Profiling

To locate computational hotspots, function-level profiling was performed using perf record.

```
perf record ./recommender
perf report
```

3.3 Software-Level Timing

Two levels of timing were implemented mainly using clock() :

- Per recommender call timing
- Batch timing across users/runs for benchmarking

4. Bottlenecks in the Original Implementation

The original implementation exhibited several performance inefficiencies primarily due to its data representation and algorithmic design choices.

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 360994511
Overhead  Command      Shared Object        Symbol
  20.68%  recommender  recommender          [.] pearson_correlation
  15.32%  recommender  recommender          [.] normalize_matrix
   9.48%  recommender  recommender          [.] calc_similarity
   8.91%  recommender  recommender          [.] calc_average
   5.18%  recommender  libc.so.6            [.] __strtok_r
   4.77%  recommender  libc.so.6            [.] __strcspn_sse42
   4.59%  recommender  libc.so.6            [.] __strspn_sse42
   3.76%  recommender  recommender          [.] calc_distance
   3.19%  recommender  [unknown]            [k] 0xffffffff932013a0
   2.60%  recommender  recommender          [.] choose_all_clusters_from_distances
   2.36%  recommender  libc.so.6            [.] __GI_____strtoll_l_internal
   2.16%  recommender  libc.so.6            [.] __GI_____strtod_l_internal
   1.73%  recommender  recommender          [.] calc_all_distances
   1.51%  recommender  libc.so.6            [.] _IO_fgets
   1.35%  recommender  libc.so.6            [.] _IO_getline_info
   1.34%  recommender  recommender          [.] get_utility_matrix
   1.27%  recommender  libc.so.6            [.] __memchr_evex
   0.90%  recommender  libc.so.6            [.] __memmove_evex_unaligned_erms
   0.64%  recommender  libc.so.6            [.] __strcpy_evex
   0.61%  recommender  libc.so.6            [.] atoi
   0.60%  recommender  libc.so.6            [.] strtok
   0.52%  recommender  libc.so.6            [.] round_and_return
   0.45%  recommender  libc.so.6            [.] _IO_file_underflow@@GLIBC_2.2.5
   0.41%  recommender  recommender          [.] new_user_movies
   0.37%  recommender  libc.so.6            [.] str_to_mpn.isra.0
   0.35%  recommender  libc.so.6            [.] __isoc23_strtol
   0.35%  recommender  recommender          [.] strtok@plt
   0.32%  recommender  libc.so.6            [.] strcspn@plt
   0.31%  recommender  libc.so.6            [.] strspn@plt
   0.29%  recommender  libc.so.6            [.] __mpn_lshift
```

4.1 Dense Utility Matrix

The system used a dense utility matrix of size (movies × users) to store user ratings. While conceptually simple, this approach is inefficient for real-world recommender systems, where the rating matrix is highly sparse and most entries are zero. As a result, a significant amount of memory was wasted storing unused values, and traversal of the full matrix introduced unnecessary computational overhead. In addition, this structure led to poor cache locality, further degrading performance during large-scale computations.

4.2 K-Means Clustering Overhead

The system used K-Means clustering on similarity scores as part of its recommendation pipeline. Although clustering can be useful for grouping similar users or items, it introduced additional computational complexity. The repeated clustering operations increased overhead, making the system less efficient, especially under large datasets and repeated execution during benchmarking.

4.3 Rating Prediction Bottleneck

A bottleneck was identified in the rating prediction function, for each movie, each similar user is scanned, and for each similar user their rated list is linearly scanned to find that movie. That is roughly O(movies × similar_users × avg_ratings_per_user).

5. Optimization Strategies Implemented

5.1 Sparse Matrix Representation

The original dense utility matrix was replaced with a sparse representation. Instead of allocating memory for all possible user x movie pairs, the optimized approach stores only non-zero ratings. This significantly reduces memory consumption and improves cache locality, as computations are performed only on relevant data rather than iterating over large regions of zero values.

```
// Original implementation
double *utility_matrix = malloc(sizeof(double) * No_of_users * No_of_movies);
// e.g., 671 users × 9125 movies = ~49 MB of doubles

// Optimized implementation
RatingEntry  { int movie; double rating; }   // one per actual rating
UserRatings  { RatingEntry *entries; int size, cap; double mean; }
RatingsStore { UserRatings *users; int no_of_users, no_of_movies; }
```

5.2 Sparse Normalization

Normalization was optimized by applying it only to existing (non-zero) ratings rather than across the entire matrix. In the original dense approach, normalization would unnecessarily process zero entries, leading to wasted computation. By restricting normalization to actual ratings, the system eliminates redundant operations and improves overall efficiency.

5.3 Removal of K-Means Clustering

The use of K-Means clustering on similarity scores was removed and replaced with a direct top-k neighbor selection strategy. While clustering introduced additional computational overhead, it did not provide sufficient benefit to justify its cost in this context. The new approach simplifies the recommendation pipeline and reduces execution time by directly identifying the most similar users without intermediate clustering.

5.4 Sparse Aware Similarity Computation

Similarity computation was optimized by leveraging the sparse data structure. Instead of comparing full rating vectors, the Pearson correlation is computed only over movies that both users have rated. This avoids unnecessary iterations over unrated items and significantly reduces the computational complexity of similarity calculations.

5.5 Efficient Prediction Strategy

The prediction function was redesigned to improve efficiency by minimizing redundant computations. Instead of iterating over all movies multiple times, the optimized approach processes each similar user's rated movies in a single pass. During this process, intermediate values such as accumulated weighted ratings, normalization factors, and counts are updated directly. A final pass over the candidate movies is then used to compute

the predicted ratings. This restructuring reduces repeated work and typically provides one of the most significant performance improvements in the system.

5.6 Optimized Sorting for Recommendations

Since only a small number of top recommendations (e.g., k = 10) are required, a simple insertion sort was used instead of more complex sorting algorithms. For such small values of k, insertion sort performs efficiently due to its low overhead, minimal data movement, and strong cache locality. This approach avoids the additional complexity and overhead of heap-based or full sorting methods while still providing fast and reliable ordering of the top results.

5.7 Parallelization of Computation

Key computational components of the system were parallelized to further improve performance. Similarity computation (Pearson correlation for each user) and prediction accumulation for candidate movies are independent operations and therefore well-suited for parallel execution. Using OpenMP pragmas, these loops were parallelized, enabling the system to utilize multiple CPU cores. This results in near-linear speedups in these stages, significantly reducing overall execution time for large datasets.

```
// Compiler flags for optimization
gcc ui.c kmeans.c matrix_normalization.c pearsons.c predictions.c
recommender.c sorting.c utility_matrix.c -lm -o recommender -O3 -fopenmp
-march=native -ffast-math
```

```
Samples: 2K of event 'cycles:Pu', Event count (approx.): 287411799
Overhead  Command      Shared Object         Symbol
  32.33%  recommender  libgomp.so.1.0.0      [.] 0x00000000000256c2
  30.17%  recommender  libgomp.so.1.0.0      [.] 0x00000000000258a2
   5.08%  recommender  libc.so.6             [.] __strtok_r
   4.10%  recommender  libc.so.6             [.] __strcspn_sse42
   4.09%  recommender  libc.so.6             [.] __strspn_sse42
   2.33%  recommender  libgomp.so.1.0.0      [.] 0x00000000000256c6
   2.23%  recommender  libc.so.6             [.] __GI______strtod_l_internal
   1.67%  recommender  libc.so.6             [.] __GI______strtoll_l_internal
   1.45%  recommender  [unknown]             [k] 0xffffffff862013a0
   1.44%  recommender  libc.so.6             [.] _IO_fgets
   1.18%  recommender  libc.so.6             [.] _IO_getline_info
   1.14%  recommender  libgomp.so.1.0.0      [.] 0x00000000000258a6
   0.96%  recommender  libc.so.6             [.] __memchr_evex
   0.93%  recommender  recommender           [.] load_ratings
   0.81%  recommender  libc.so.6             [.] __memmove_evex_unaligned_erms
   0.78%  recommender  libc.so.6             [.] __GI_strtoll
   0.68%  recommender  libc.so.6             [.] __strcpy_evex
   0.62%  recommender  recommender           [.] make_prediction_new._omp_fn.0
   0.59%  recommender  libgomp.so.1.0.0      [.] 0x0000000000024f42
   0.56%  recommender  libc.so.6             [.] _IO_file_underflow@@GLIBC_2.2.5
   0.55%  recommender  libgomp.so.1.0.0      [.] 0x00000000000256c0
   0.49%  recommender  libc.so.6             [.] str_to_mpn.isra.0
   0.46%  recommender  libgomp.so.1.0.0      [.] 0x00000000000256cd
   0.42%  recommender  libc.so.6             [.] round_and_return
   0.40%  recommender  libc.so.6             [.] strtok
   0.33%  recommender  libc.so.6             [.] strspn@plt
   0.33%  recommender  libgomp.so.1.0.0      [.] 0x00000000000256cb
   0.31%  recommender  libgomp.so.1.0.0      [.] 0x00000000000258af
   0.27%  recommender  recommender           [.] strtok@plt
   0.27%  recommender  libc.so.6             [.] __mpn_lshift
   0.22%  recommender  recommender           [.] findusers
   0.21%  recommender  libc.so.6             [.] _IO_switch_to_get_mode
   0.21%  recommender  recommender           [.] normalize_ratings_store
```

6. Performance Comparison

Processing time taken for 5 runs

	1	2	3	4	5	Sum	Average
Before (s)	0.21	0.18	0.20	0.20	0.20	0.99	0.198
After (s)	0.08	0.21	0.09	0.07	0.21	0.66	0.132

Metric	**Before**	**After**
CPU Cycles	397 M	364 M
Instructions Executed	1058 M	281 M
Instructions per Cycle (IPC)	2.66	0.77
Page Faults	28,000	10,000
Branch Misses	1.2 M	0.8 M
Time elapsed for 5 runs (s)	1.43	1.06

Before optimization:

```
Performance counter stats for './recommender' (5 runs):

          278.64 msec task-clock:u                     #    0.195 CPUs utilized              ( +-  2.01% )
               0      context-switches:u               #    0.000 /sec
               0      cpu-migrations:u                 #    0.000 /sec
           28786      page-faults:u                    #  103.310 K/sec                      ( +-  0.00% )
       397803878      cycles:u                         #    1.428 GHz                          ( +-  0.13% )
      1057813448      instructions:u                   #    2.66  insn per cycle               ( +-  0.00% )
       135279760      branches:u                       #  485.507 M/sec                        ( +-  0.00% )
         1242935      branch-misses:u                  #    0.92% of all branches              ( +-  0.87% )
                      TopdownL1                 #      6.4 %  tma_backend_bound
                                                #     71.0 %  tma_bad_speculation
                                                #      4.9 %  tma_frontend_bound
                                                #     17.7 %  tma_retiring             ( +-  0.10% )

          1.4263 +- 0.0154 seconds time elapsed  ( +-  1.08% )
```

After optimization:

```
Performance counter stats for './recommender' (5 runs):

          216.20 msec task-clock:u                     #    0.205 CPUs utilized              ( +- 15.25% )
               0      context-switches:u               #    0.000 /sec
               0      cpu-migrations:u                 #    0.000 /sec
           10072      page-faults:u                    #   46.588 K/sec                      ( +-  0.01% )
       363998453      cycles:u                         #    1.684 GHz                         ( +- 34.14% )
       281152331      instructions:u                   #    0.77  insn per cycle              ( +-  1.93% )
        65518134      branches:u                       #  303.051 M/sec                       ( +-  2.37% )
          816749      branch-misses:u                  #    1.25% of all branches             ( +-  1.71% )
                      TopdownL1                 #     42.2 %  tma_backend_bound
                                                #     49.4 %  tma_bad_speculation
                                                #      6.9 %  tma_frontend_bound
                                                #      1.6 %  tma_retiring             ( +- 27.20% )

          1.0555 +- 0.0165 seconds time elapsed  ( +-  1.56% )
```

7. Analysis of Results

The optimized version of the system demonstrated significant improvement in overall performance, particularly in terms of computational efficiency and memory behavior. The number of CPU cycles decreased from 397 million to 364 million, indicating a reduction in CPU work. Moreover, the total number of executed instructions dropped dramatically from 1058 million to 281 million, reflecting significant simplification of computation and elimination of redundant operations. Although the Instructions Per Cycle (IPC) decreased from 2.66 to 0.77, this does not necessarily indicate worse performance; rather, it suggests that the optimized version executes fewer, more targeted instructions, with less opportunity for instruction-level parallelism.

Page faults were reduced from approximately 28,000 to 10,000, indicating better memory locality and more efficient data access patterns, likely due to the use of sparse data structures. Similarly, branch misses decreased from 1.2 million to 0.8 million, suggesting improved control flow and more predictable execution paths.

The overall elapsed time decreased from 1.43 seconds to 1.06 seconds, while the CPU processing time (as measured by clock()) reduced from 0.198 seconds to 0.132 seconds. This confirms that the optimizations not only reduced computational overhead but also improved execution time.

8. Conclusion

The optimized recommender system transitions from a dense, computationally heavy approach to a sparse, efficient collaborative filtering model. Through the new optimized approach, computational complexity has been reduced and memory efficiency has been improved while eliminating unnecessary overheads and using parallelization.